Project Title: EduPulse AI – Intelligent Multi-Tenant Compliance & Audit Assistant

EduPulse AI is an advanced compliance and internal audit platform designed for highly regulated industries (such as Fintech, Healthcare, or Higher Education). It allows companies to upload massive, unstructured regulatory frameworks, internal policies, and historical audit reports. Staff can then query the system using natural language to perform complex gap analyses, check policy compliance, and generate audit-ready documentation.

Core Architecture & Tech Stack

- **Backend Framework:** Spring Boot 3.x & Java 21 (leveraging Virtual Threads for high-throughput document parsing).
- **AI Orchestration:** Spring AI (v2.0+) utilizing the unified `ChatClient` and `VectorStore` APIs.
- **Vector Database: PostgreSQL with the `pgvector` extension**, handling relational business data and high-dimensional semantic embeddings side-by-side.
- **LLM Providers:** Orchestrated via Spring AI starters (e.g., OpenAI `gpt-4o` for reasoning, `text-embedding-3-small` for embeddings) or local execution using Ollama (`llama3.2`)

Implementing "Advanced RAG" with Spring AI

A basic RAG application simply turns a query into an embedding, searches a database, and feeds the top blocks to an LLM. **EduPulse AI** upgrades this into an **Advanced RAG** pipeline using native Spring AI components

1. Pre-Retrieval: Intelligent Ingestion & Multi-Query Expansion

- **Hierarchical Chunking:** Instead of naive token splitting, use a customized `TokenTextSplitter` combined with parent-child chunking. Store small child chunks (e.g., 200 tokens) for precise vector matching, but map them to larger parent paragraphs (e.g., 1000 tokens) to supply rich context to the LLM.
- **Query Expansion via `MultiQueryExpander`:** Compliance questions are often poorly phrased. Spring AI's `MultiQueryExpander` rewrites a single user prompt into 3-4 variations from different technical angles, executing parallel vector searches to maximize recall.

2. Retrieval: Hybrid Search & Metadata Guardrails

- **Hybrid Search via Postgres:** Execute a combined search using `pgvector` HNSW index similarity search for semantic meaning, paired with classic PostgreSQL `tsvector` full-text search for exact regulatory codes
- **Metadata Filtering:** Ensure strict data isolation. Utilize Spring AI's SQL-like expression filters (`Filter.Expression`) to ensure a user only queries documents belonging to their specific organizational unit or security clearance level.

3. Post-Retrieval: Reranking & Context Compression

- **Reranking:** Pass the top 20 retrieved chunks through a Reranker advisor (such as Cohere or a local cross-encoder) to re-order documents based on exact relevance before prompt injection.
- **Compression (`CompressionQueryTransformer`):** To minimize token costs and eliminate noise, use context compression to distill the retrieved text blocks into only the most relevant sentences answering the specific user prompt.

4. Generation & Guardrails

- **`RetrievalAugmentationAdvisor`:** Wire the entire pipeline elegantly into Spring AI's modular client execution.
- **Safety Guardrails:** Use `SafeguardAdvisor` and `TopicBoundaryAdvisor`

to prevent the bot from hallucinatory advice or answering questions unrelated to corporate compliance.

Key Enterprise Features to Build

1. **Automated Gap Analysis (Agentic Workflow):** A feature where users upload a *new* regulation PDF, and the application automatically breaks it down chunk-by-chunk, queries the existing internal policy vector database, and generates a structured report highlighting compliance gaps.
2. **Audit Trail & Citations:** Every answer generated must return the exact source document name, page number, and paragraph snippet stored within the `pgvector` metadata payload, ensuring high audit transparency.
3. **Asynchronous Document Ingestion:** Use Spring Boot's `@Async` or job schedulers like JobRunr to handle the heavy lifting of extracting, embedding, and loading multi-gigabyte regulatory libraries without blocking the user interface.

Functional Requirements (FR)

Document Processing & Ingestion

- **Multi-Format Ingestion:** The system must accept and process regulatory documents in PDF, DOCX, and TXT formats.
- **Hierarchical Document Chunking:** The ingestion service must split documents into small child chunks for vector matching while preserving parent sections for context generation.
- **Asynchronous Parsing:** Document uploads exceeding 10MB must be processed asynchronously using a background worker pattern without blocking the UI.

Search & Retrieval (Advanced RAG)

- **Hybrid Search Execution:** The system must combine semantic search queries using `pgvector` with exact keyword matching using PostgreSQL full-text search (`tsvector`).
- **Multi-Query Expansion:** The platform must automatically generate 3 variations of an ambiguous user query to maximize vector document retrieval recall.
- **Strict Source Citations:** Every AI-generated response must cite the source document name, page number, and original text snippet.

Access Control & Tenancy

- **Multi-Tenant Data Isolation:** The system must use metadata-driven filtering to ensure users can only query documents belonging to their assigned organization unit.
- **Granular Role-Based Access (RBAC):** The platform must restrict document uploading functions to users with the `ROLE_ADMIN` or `ROLE_COMPLIANCE_OFFICER` roles.

Audit & Analytics

- **Automated Gap Analysis Reports:** Users must be able to request an automated compliance comparison between a newly uploaded document and existing guidelines, yielding an exportable evaluation.
- **Conversation History Tracking:** The system must retain an audit log of all compliance queries submitted by users, along with the specific context chunks used to generate the responses.

Non-Functional Requirements (NFR)

Performance & Scalability

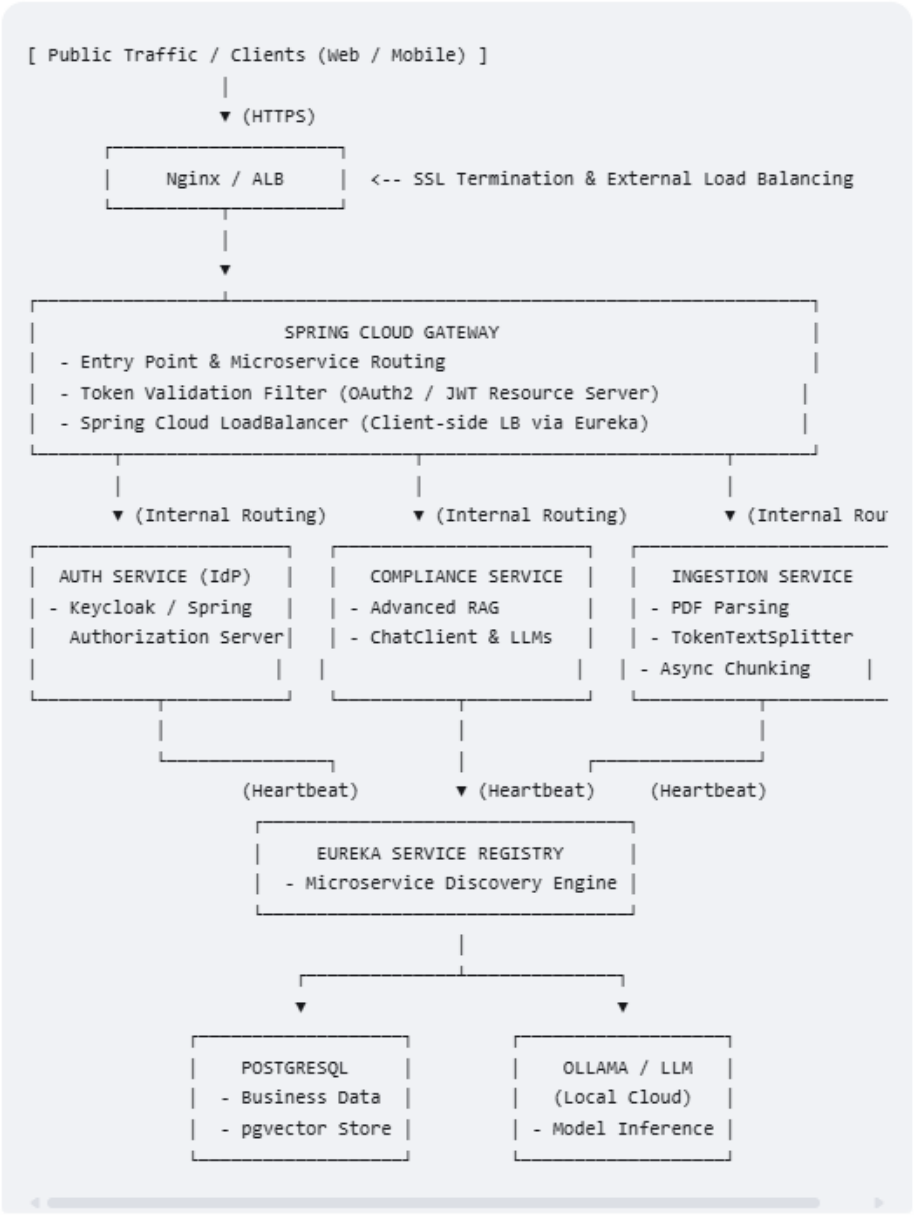
- **Response Latency:** The system must complete semantic vector lookup queries and initiate LLM streaming generation within less than 2.0 seconds under standard loads.
- **Concurrent Ingestion Handling:** The microservice architecture must scale dynamically to process up to 50 simultaneous multi-page document ingestion pipelines without dropping requests.

Security & Compliance

- **Stateless Authentication:** All inter-service communications must be secured and authorized statelesssly via JWT tokens validated through the API Gateway.
- **Data-at-Rest Encryption:** All stored business profiles, regulatory data, and text embeddings held within PostgreSQL must be encrypted at rest using AES-256.

Reliability & Availability

- **Service Registry Resilience:** The Eureka Service Registry must support a cluster configuration ensuring 99.9% uptime, enabling seamless routing and failover across healthy microservice nodes.


[Public Traffic / Clients (Web / Mobile)]
(HTTPS)
Nginx / ALB
<-- SSL Termination & External Load Balancing
SPRING CLOUD GATEWAY
- Entry Point & Microservice Routing
- Token Validation Filter (OAuth2 / JWT Resource Server)
- Spring Cloud LoadBalancer (Client-side LB via Eureka)
(Internal Routing)
(Internal Routing)
(Internal Rou
AUTH SERVICE (IdP)
- Keycloak / Spring Authorization Server
COMPLIANCE SERVICE
- Advanced RAG
- ChatClient & LLMs
INGESTION SERVICE
- PDF Parsing
- TokenTextSplitter
- Async Chunking
(Heartbeat)
(Heartbeat)
(Heartbeat)
EUREKA SERVICE REGISTRY
- Microservice Discovery Engine
POSTGRESQL
- Business Data
- pgvector Store
OLLAMA / LLM
(Local Cloud)
- Model Inference